

Pseudocodice della codebase

Reward Learning e Imitation Learning per la guida autonoma in SUMO

sumo-human-feedback-rl

19 giugno 2026

Questo documento riporta lo pseudocodice formale dei tre algoritmi di apprendimento implementati nel package `human_feedback_rl`:

- **Algoritmo 1** — loop esterno generico di *reward learning* (classe `BaseRewardLearningAlgorithm`), condiviso dagli algoritmi a feedback.
- **Algoritmo 2** — apprendimento da *preferenze* con perdita Bradley–Terry (`PreferenceAlgorithm`, Christiano et al. 2017).
- **Algoritmo 3** — apprendimento da *dimostrazioni* via Maximum-Entropy IRL (`DemoAlgorithm`).
- **Algoritmo 4** — imitation learning iterativo *Dagger* con behaviour cloning (`DaggerAlgorithm`).
- **Sezione 5** — le classi helper riusabili di `common/` (rete di reward, fragmenters, gatherers, dataset, generatore di traiettorie, wrapper d’ambiente, replay buffer).

Notazione e strutture dati

Una *transizione* è la tupla $t = (o, a, r^{\text{true}}, \mathbf{s}', d)$ con osservazione o , azione a , reward vero d’ambiente r^{true} , stato terminale one-hot $\mathbf{s}' \in \{0, 1\}^7$ (*arrived*, *collided*, *off_road*, *timeout*, *running*, *teleported*, *removed*) e flag di fine episodio d . Una *traiettoria* $\tau = (t_1, \dots, t_T)$ è una sequenza di transizioni.

Il **reward model** $\tilde{r}_{\Theta}$ è un *ensemble* di K reti $\{\text{member}_k\}_{k=1}^K$, ciascuna un `NormalizedRewardNet` che avvolge una MLP `SumoRewardNet`; l’intero ensemble è a sua volta normalizzato (double-wrapping). Il return di una traiettoria secondo il membro k è $R_k(\tau) = \sum_{t \in \tau} \text{member}_k(o_t, a_t, \mathbf{s}'_t, d_t)$, e la predizione usata dalla policy è $\tilde{r}_{\Theta}.\text{predict}(o, a, \mathbf{s}', d) = (\frac{1}{K} \sum_k \text{member}_k - \mu_{\text{out}}) / (\sigma_{\text{out}} + \varepsilon)$.

1. Loop esterno di reward learning

Algorithm 1 BASEREWARDLEARNINGALGORITHM.TRAIN — loop esterno generico

Input: ambiente $\mathcal{E}$, agente π_ϕ , reward model $\tilde{r}_\Theta$, timesteps totali T_{tot} , timesteps/iter. T , query totali Q , schedule delle query

Output: agente addestrato π_ϕ

```

1:  $N \leftarrow \lfloor T_{\text{tot}}/T \rfloor$ 
2:  $\text{schedule} \leftarrow \text{BUILDQUERYSCCHEDULE}(N, Q)$   $\triangleright$  query per iterazione
3:  $\text{BEFORETRAIN}()$   $\triangleright$  hook: eventuale pre-warmup
4: for  $q \in \text{schedule}$  do  $\triangleright$  una entry per iterazione
5:   if  $q = 0$  then continue
6:   end if
   — Raccolta dati —
7:    $T_\varepsilon \leftarrow \lfloor \text{exploration\_frac} \cdot T \rfloor$ 
8:    $\mathcal{T}^M \leftarrow \text{SAMPLEROLLOUT}(\pi_\phi, \mathcal{E}, T, T_\varepsilon)$   $\triangleright$  traiettorie fresche dalla policy
9:    $(\mathcal{F}, \mathcal{Y}) \leftarrow \text{COLLECTFEEDBACK}(q)$   $\triangleright$  frammenti + etichette (hook astratto)
10:   $\text{PUSHDATA}(\mathcal{F}, \mathcal{Y})$   $\triangleright$  split  $\mathcal{D}_{\text{train}}/\mathcal{D}_{\text{val}}$  (hook astratto)
   — Addestramento reward model —
11:   $\text{BEFOREREWARDTRAINING}()$   $\triangleright$  log correlazione RM vs. ground-truth
12:   $\text{TRAINREWARDMODEL}()$   $\triangleright$  aggiornamenti gradiente (hook astratto)
   — Addestramento agente —
13:   $\text{BEFOREAGENTTRAINING}()$   $\triangleright$  hook: normalizzazione reward
14:   $\text{TRAINAGENT}(\pi_\phi, \tilde{r}_\Theta, \mathcal{E}, T)$   $\triangleright$  PPO/SAC con reward appreso
   — Logging & checkpoint —
15:   $\text{LOGITERATION}()$ ; if checkpoint dovuto then  $\text{SAVECHECKPOINT}()$ 
16:   $\text{iteration} \leftarrow \text{iteration} + 1$ 
17: end for
18: return  $\pi_\phi$ 

```

Schedule delle query. Con pesi $w(t)$ scelti tra **constant** $w(t) = 1$, **hyperbolic** $w(t) = \frac{1}{1+t}$, **inverse_quadratic** $w(t) = \frac{1}{1+t^2}$ valutati su $t \in \text{linspace}(0, 1, N)$, le query sono distribuite proporzionalmente a $p = w / \sum w$, con **initial_queries** assegnate all'iterazione 0.

2. Apprendimento da preferenze (Bradley–Terry)

Algorithm 2 PREFERENCEALGORITHM.TRAIN_REWARD_MODEL — perdita Bradley–Terry

Input: dataset di preferenze $\mathcal{D}_{\text{train}}$, ensemble $\{\text{member}_k\}_{k=1}^K$ con ottimizzatori $\{\text{opt}_k\}$, gradient steps G , batch size B

Implementa COLLECTFEEDBACK/PUSHDATA/TRAINREWARDMODEL dell'Alg. 1.

```

// COLLECTFEEDBACK(q): genera coppie di frammenti ed etichette di preferenza
1:  $\mathcal{F} \leftarrow \text{FRAGMENTER}(\mathcal{T}^M, \text{fragment\_length}, q)$   $\triangleright$  random o active (high-variance)
2:  $\mathcal{Y} \leftarrow \text{PREFERENCEGATHERER}(\mathcal{F})$   $\triangleright$  preferenza sintetica dal reward vero

// TRAINREWARDMODEL: aggiorna ogni membro su un bootstrap di  $\mathcal{D}_{\text{train}}$ 
3: for  $k = 1, \dots, K$  do  $\triangleright$  un thread per membro
4:    $\mathcal{B}_k \leftarrow \mathcal{D}_{\text{train}}.\text{bootstrap}()$   $\triangleright$  campionamento con rimessa
5:   for  $\text{step} = 1, \dots, G$  do
6:     estrai un mini-batch di  $B$  coppie  $(\sigma^1, \sigma^2)$  con etichette  $(y_1, y_2)$  da  $\mathcal{B}_k$ 
7:      $r_1 \leftarrow \bar{R}_k(\sigma^1), \quad r_2 \leftarrow \bar{R}_k(\sigma^2)$   $\triangleright$  reward medio del frammento
8:      $p_1 \leftarrow \sigma(r_1 - r_2), \quad \mathbf{p} \leftarrow [p_1, 1 - p_1]$   $\triangleright$  modello Bradley–Terry
9:      $\mathcal{L}_k \leftarrow -\text{mean}[y_1 \log p_1 + y_2 \log(1 - p_1)]$   $\triangleright$  cross-entropy
10:     $\text{opt}_k: \boldsymbol{\theta}_k \leftarrow \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}_k} \mathcal{L}_k$   $\triangleright$  Adam, weight decay  $\lambda$ 
11:   end for
12: end for
13: valuta loss/accuracy su  $\mathcal{D}_{\text{train}}$  e  $\mathcal{D}_{\text{val}}$  e registra le metriche

```

```

// BEFOREAGENTTRAINING: ricentra il reward sui dati del rollout corrente
14: per ogni membro  $k$ :  $\mu_k \leftarrow \text{mean}[\text{member}_k^{\text{raw}}(\mathcal{T}^M)]$ ; poi  $\mu_{\text{out}} \leftarrow \text{mean}[\hat{r}_{\boldsymbol{\Theta}}^{\text{raw}}(\mathcal{T}^M)]$ 

```

La perdita complessiva minimizzata è, su un batch di coppie con etichette $y \in \{0, 1\}$ (oppure soft):

$$\mathcal{L}_{\text{BT}} = -\mathbb{E}_{(\sigma^1, \sigma^2, y)} \left[y \log \sigma(\bar{R}(\sigma^1) - \bar{R}(\sigma^2)) + (1 - y) \log \sigma(\bar{R}(\sigma^2) - \bar{R}(\sigma^1)) \right].$$

3. Apprendimento da dimostrazioni (MaxEnt IRL)

Algorithm 3 DEMOALGORITHM.TRAIN — MaxEnt IRL da dimostrazioni fisse

Input: ambiente $\mathcal{E}$, agente π_ϕ (PPO/SAC), traiettorie esperto fisse $\mathcal{T}^E$, reward model $\tilde{r}_\Theta$, perdita $\ell \in \{\text{maxent}, \text{maxent_2}, \text{demo}, \text{maxent_corrected}, \text{demo_corrected}\}$

Input: gradient steps G , batch esperto n_e , batch modello n_m , temperatura T_{emp}

Output: agente addestrato π_ϕ

```

1: if initial_agent_timesteps  $T_0 > 0$  then                                 $\triangleright$  bootstrap del reward prima della policy
2:    $\mathcal{T}^M \leftarrow \text{SAMPLEROLLOUT}(\pi_\phi, \mathcal{E}, T_0);$                                  $\text{TRAINREWARDMODEL}();$ 
    $\text{TRAINAGENT}(\pi_\phi, \tilde{r}_\Theta, \mathcal{E}, T_0)$ 
3: end if
4: for iter = 0, ...,  $N - 1$  do
5:    $\mathcal{T}^M \leftarrow \text{SAMPLEROLLOUT}(\pi_\phi, \mathcal{E}, T, T_\varepsilon)$                                  $\triangleright$  traiettorie fresche
6:    $\text{LOGVALIDATION}(\mathcal{T}^M, \text{"pre\_update"})$                                  $\triangleright$  Spearman  $\rho$ , gap arrived/collided
7:    $\text{TRAINREWARDMODEL}()$                                  $\triangleright$  vedi sotto
8:    $\text{LOGVALIDATION}(\mathcal{T}^M, \text{"post\_update"})$ 
9:    $\text{TRAINAGENT}(\pi_\phi, \tilde{r}_\Theta, \mathcal{E}, T)$                                  $\triangleright$  reward relabeling per off-policy
10:   $\text{LOGITERATION}();$  if checkpoint then  $\text{SAVECHECKPOINT}()$ 
11: end for
12: return  $\pi_\phi$ 

```

// TRAINREWARDMODEL : per ogni membro k , G passi su ℓ

```

13: for  $k = 1, \dots, K$  do
14:   for step = 1, ...,  $G$  do
15:      $I_E \sim \text{Unif}\{\mathcal{T}^E\}^{n_e}, I_M \sim \text{Unif}\{\mathcal{T}^M\}^{n_m}$                                  $\triangleright$  senza rimessa
16:      $\mathbf{R}^E \leftarrow [R_k(\tau)]_{\tau \in I_E}, \mathbf{R}^M \leftarrow [R_k(\tau)]_{\tau \in I_M}$ 
17:      $\mathcal{L}_k \leftarrow \text{REWARDLOSS}_\ell(\mathbf{R}^E, \mathbf{R}^M)$                                  $\triangleright$  vedi tabella perdite
18:     opt $_k$ :  $\theta_k \leftarrow \theta_k - \eta \nabla \mathcal{L}_k$                                  $\triangleright$  Adam + weight decay
19:   end for
20: end for

```

Varianti della perdita REWARDLOSS_ℓ (con $\text{lse} = \text{logsumexp}$):

$$\begin{aligned}
\text{maxent} : & \quad -\overline{\mathbf{R}^E} + \text{lse}(\mathbf{R}^M) - \log n_m \\
\text{maxent_2} : & \quad -\overline{\mathbf{R}^E} + \text{lse}(\mathbf{R}^M \cup \mathbf{R}^E) - \log(n_e + n_m) \\
\text{demo} : & \quad -\overline{\mathbf{R}^E} + \overline{\mathbf{R}^M} \quad (\text{differenza delle medie}) \\
\text{maxent_corrected} : & \quad -\frac{1}{T_{emp}} \overline{\mathbf{R}^E} + \text{lse}\left(\frac{1}{T_{emp}} \mathbf{R}^M - \log q(\tau)\right) - \log n_m \quad (\text{correzione importance sampling}) \\
\text{demo_corrected} : & \quad \text{mean softplus}\left(-(\overline{R}_i^E - \overline{R}_i^M)/T_{emp}\right) \quad (\text{ranking bounded, reward medio per passo})
\end{aligned}$$

dove $q(\tau)$ è la log-probabilità della policy di proposta sulla traiettoria τ (memorizzata nelle transizioni del rollout).

4. Imitation learning iterativo (Dagger)

Algorithm 4 DAGGERALGORITHM.TRAIN — DAgger con behaviour cloning

Input: ambiente $\mathcal{E}$, agente π_ϕ , esperto π^E , round N_r , episodi/round M , epoche BC E , batch size B , decadimento β_{dec}

Output: agente addestrato π_ϕ

```

1:  $\mathcal{D} \leftarrow \emptyset$   $\triangleright$  dataset aggregato di transizioni esperto-etichettate
2: for round = 0, ...,  $N_r - 1$  do
3:    $\beta \leftarrow \beta_{\text{dec}}^{\text{round}}$   $\triangleright \beta=1$  al round 0 (puro esperto), poi  $\rightarrow 0$ 
   — 1) Raccolta traiettorie miscelando esperto e agente —
4:   for episodio = 1, ...,  $M$  do
5:      $o \leftarrow \mathcal{E}.\text{reset}()$ 
6:     while episodio non terminato do
7:        $a^{\text{ag}} \leftarrow \pi_\phi(o); a^E \leftarrow \pi^E(o)$   $\triangleright$  azione agente e azione esperto
8:        $a^{\text{exec}} \leftarrow a^E$  con prob.  $\beta$ , altrimenti  $a^{\text{ag}}$   $\triangleright$  miscela di controllo
9:        $o' \leftarrow \mathcal{E}.\text{step}(a^{\text{exec}})$ 
10:      aggiungi la transizione  $(o, a^E)$  all'episodio  $\triangleright$  etichetta = azione esperto
11:       $o \leftarrow o'$ 
12:    end while
13:  end for
  — 2) Aggregazione del dataset —
14:   $\mathcal{D} \leftarrow \mathcal{D} \cup \{\text{transizioni raccolte}\}$ 
  — 3) Behaviour cloning sull'intero  $\mathcal{D}$  —
15:  for epoca = 1, ...,  $E$  do
16:    for ogni mini-batch  $(o_b, a_b^E)$  di dimensione  $B$  in  $\mathcal{D}$  (permutato) do
17:       $\mathcal{L}_{\text{BC}} \leftarrow -\text{mean log } \pi_\phi(a_b^E | o_b)$   $\triangleright$  negative log-likelihood
18:      aggiorna  $\phi$  con Adam su  $\nabla_\phi \mathcal{L}_{\text{BC}}$ 
19:    end for
20:  end for
  — 4) Valutazione deterministica —
21:  valuta  $\pi_\phi$  (deterministico) per  $n_{\text{eval}}$  episodi; registra reward/lunghezza medi
22: end for
23: return  $\pi_\phi$ 

```

5. Classi helper (common/)

Questa sezione descrive i componenti riusabili che gli algoritmi delle sezioni precedenti compongono: rete di reward, generazione di frammenti, raccolta delle etichette, dataset, generatore di traiettorie, wrapper d'ambiente e replay buffer.

5.1 Rete di reward (reward_nets.py)

La factory `make_reward_ensemble` costruisce la gerarchia *double-wrapped* $\tilde{r}_\Theta = \text{NORM}(\text{ENSEMBLE}[\text{NORM}(\text{SUMO}$

Algorithm 5 Componenti di `reward_nets`

```
// SUMOREWARDNET.FORWARD — MLP per transizione
1:  $x \leftarrow [o \parallel a \parallel s' \parallel d]$   $\triangleright$  concatenazione, dim.  $d_o + d_a + 7 + 1$ 
2: return  $\text{MLP}(x)$   $\triangleright$  2 layer da 128, attivazione tanh, output scalare

// REWARDENSEMBLE.FORWARD — media tra i membri
3: return  $\frac{1}{K} \sum_{k=1}^K \text{member}_k(o, a, s', d)$ 
   PREDICT_ALL: restituisce la matrice  $(N \times K)$  dei reward per membro
   PREDICT_MEAN_STD: media e deviazione standard tra membri (disaccordo)

// NORMALIZEDREWARDNET — normalizzazione affine solo in inferenza
4: forward $(\cdot) =$  rete grezza  $\triangleright$  l'obiettivo di training resta non alterato
5: predict $(\cdot) = (\text{raw} - \mu) / (\sigma + \varepsilon)$   $\triangleright \mu, \sigma$  buffer persistenti
6: set_mean $(m)$ :  $\mu \leftarrow (1 - \alpha)\mu + \alpha m$   $\triangleright$  EMA; con  $\alpha=1$  assegnazione diretta
```

5.2 Fragmenters (`fragmenters.py`)

Trasformano le traiettorie del rollout in frammenti (o coppie di frammenti) da sottoporre a giudizio.

Algorithm 6 RANDOMFRAGMENTER e HIGHVARIANCEFRAGMENTER

```
// _sample_fragments( $\mathcal{T}^M, L, n$ ): campiona  $n$  frammenti di lunghezza  $L$ 
1: for  $i = 1, \dots, n$  do
2:   scegli  $\tau \in \mathcal{T}^M$  con prob.  $\propto \lfloor |\tau|/L \rfloor + 1$   $\triangleright$  traiettorie lunghe più probabili
3:    $\text{start} \sim \text{Unif}\{0, \dots, |\tau| - L\}$ ; aggiungi  $\tau[\text{start} : \text{start}+L]$ 
4: end for
   RANDOMPAIRFRAGMENTER: campiona  $2n$  frammenti e li accoppia  $(f_{2i}, f_{2i+1})$ 

// HIGHVARIANCE: campionamento attivo per disaccordo dell'ensemble
5: campiona  $\text{oversample} \cdot (\text{n. richiesti})$  frammenti candidati
6: for ogni candidato  $f$  do
7:    $v(f) \leftarrow \text{Var}_k[\sum_{t \in f} \text{member}_k(t)]$   $\triangleright$  varianza del return tra i  $K$  membri
8: end for
9: return i frammenti con  $v(f)$  massima  $\triangleright$  punti più informativi / incerti
```

5.3 Gatherers (`gatherers.py`)

Algorithm 7 Generazione delle etichette

```
// PREFERENCEGATHERERFROMREWARD: preferenza sintetica dal reward vero
1: for ogni coppia  $(\sigma^1, \sigma^2)$  do
2:    $r_1 \leftarrow \frac{1}{|\sigma^1|} \sum r^{\text{true}}, \quad r_2 \leftarrow \frac{1}{|\sigma^2|} \sum r^{\text{true}}$   $\triangleright$  reward medio per passo
3:   binary:  $y = \mathbb{I}(r_1 > r_2), \cdot$  ; soft:  $y_1 = \sigma((r_1 - r_2)/T_{\text{emp}})$ 
4:   binary_bernoulli:  $y_1 \sim \text{Bernoulli}(y_1^{\text{soft}})$ 
5: end for

// DEMOGATHERERFROMEXPERT: rietichetta le azioni con quelle dell'esperto
6: per ogni frammento  $f$ : sostituisci ogni  $a_t$  con  $\pi^E(o_t)$ , conservando  $o_t, s'_t, d_t$ 
```

5.4 Dataset (`datasets.py`)

BaseDataset è una coda (`deque`) di capacità `queue_size`; PreferenceDataset/DemonstrationDataset la specializzano per coppie (frammento, etichetta). Il **bootstrap** restituisce un nuovo dataset di

pari dimensione campionato *con rimessa* (usato per addestrare i membri dell'ensemble in modo decorrelato):

$$\mathcal{B} = \{d_i\}_{i \in I}, \quad I \sim \text{Unif}\{1, \dots, |\mathcal{D}|\}^{|\mathcal{D}|} \text{ (con rimessa).}$$

5.5 Generatore di traiettorie (trajectory_generators.py)

`TrajectoryGeneratorFromAgent` collega l'agente SB3 all'ambiente avvolto (buffering $\rightarrow$ reward $\rightarrow$ monitor) e ne raccoglie le traiettorie.

Algorithm 8 TRAJECTORYGENERATORFROMAGENT.SAMPLE e ROLLOUT_AGENT

Input: passi richiesti dall'agente T , passi di esplorazione T_ε

- 1: $\mathcal{T}^M \leftarrow$ traiettorie già completate nel buffer
- 2: **if** transizioni disponibili $< T$ **then**
- 3: `ROLLOUTAGENT`(π_ϕ , $T - \text{disponibili}$); aggiungi le nuove traiettorie completate
- 4: **end if**
- 5: **if** $T_\varepsilon > 0$ **then**
- 6: `ROLLOUTAGENT`(π_ε , T_ε) $\triangleright$ policy ε -greedy; aggiungi a $\mathcal{T}^M$
- 7: **end if**
- 8: **return** $\mathcal{T}^M$

// ROLLOUTAGENT: avanza l'env finché non raccoglie $\geq$ steps e chiude gli episodi attivi

- 9: **while** vero **do**
 - 10: $a \leftarrow \pi_\phi.\text{predict}(o)$; memorizza $\log \pi_\phi(a | o)$ nelle transizioni
 - 11: $o, \cdot, \text{done} \leftarrow \mathcal{E}.\text{step}(a)$; `collected` $+= n_{\text{env}}$
 - 12: **if** `collected` $\geq$ steps **then** marca gli env come “in chiusura” e termina quando tutti done
 - 13: **end if**
 - 14: **end while**
-

POLICY_ACTION_LOG_PROBS. Calcola $\log \pi(a | o)$ per PPO (categorica o gaussiana *tanh-squashed* con massa agli estremi via $\log \Phi$) e per SAC (densità gaussiana riportata nelle coordinate d'azione dell'ambiente, sottraendo $\sum \log(\text{action_scale})$); usata dalla correzione di importance sampling di `maxent_corrected`.

5.6 Wrapper d'ambiente (env_wrappers.py)

Algorithm 9 Wrapper vettorizzati dell'ambiente

// ENVREWARDWRAPPER.STEP: sostituisce il reward vero con quello appreso

- 1: $o', r^{\text{true}}, \text{done}, \text{info} \leftarrow \mathcal{E}.\text{step}(a)$
- 2: $s' \leftarrow \text{ONEHOT}(\text{info}[\text{ego_status}])$
- 3: **return** (o' , $\tilde{r}_\Theta.\text{predict}(o_{\text{pre}}, a, s', \text{done})$, `done`, `info`) $\triangleright$ usa l'obs pre-step

// ENVBUFFERINGWRAPPER.STEP: registra una TRANSITION per env

- 4: per ogni env i con maschera attiva: aggiungi ($o_i, a_i, r_i^{\text{true}}, s'_i, d_i, \log \pi_i$) alla traiettoria parziale
- 5: se d_i : sposta la traiettoria in “completate” e ripartila

// POLICYEXPLORATIONWRAPPER.PREDICT: ε -greedy

- 6: con prob. ε : azione casuale dallo `action_space`; altrimenti $\pi_\phi(o)$
 - 7: **action_log_prob**: $\log[(1 - \varepsilon) \pi(a|o) + \varepsilon \text{Unif}]$ $\triangleright$ densità della mistura
-

La classe `_RunningMeanStd` mantiene media e varianza online con l'algoritmo di *Welford*.

5.7 Replay buffer con relabeling (replay_buffers.py)

`RewardRelabelReplayBuffer` estende il `ReplayBuffer` di SB3 per gli algoritmi off-policy (SAC): memorizza anche lo stato terminale one-hot e, al campionamento, *ricalcola* i reward con il reward model corrente.

Algorithm 10 REWARDRELABELREPLAYBUFFER. `__GET__SAMPLES`

```
1: campiona indici di batch dalle transizioni memorizzate
2: if relabel_rewards then
3:    $r \leftarrow \tilde{r}_{\Theta}.\text{predict}(o, \text{UNSCALE}(a), s', d)$   $\triangleright$  reward aggiornato al RM corrente
4: else
5:    $r \leftarrow$  reward memorizzato
6: end if
7: return campioni SB3 ( $o, a, o', d(1-\text{timeout}), r$ )
   SAMPLE_REWARD_STALENESS: confronta reward memorizzati vs. correnti (diagnostica)
```

Schedule del learning rate (`schedules.py`). `InverseSchedule` interpola tra valore iniziale e finale con decadimento iperbolico $1/(1 + \kappa p)$ normalizzato, in funzione del progresso $p \in [0, 1]$.

Riepilogo dei tre paradigmi

Algoritmo	Segnale di supervisione	Modello appreso	Perdita
Preference	preferenze su coppie di frammenti	reward model (ensemble)	Bradley–Terry (cross-entropy)
Demo	traiettorie esperte fisse	reward model (ensemble)	MaxEnt IRL / ranking
DAgger	azioni esperte (DAgger)	policy diretta	NLL (behaviour cloning)

Documento generato a partire dai sorgenti `human_feedback_rl/algorithms/{demo,preference,dagger}_algorithm.py` e `common/base_reward_learning_algorithm.py`.